

任务 9：重构登录逻辑并接入 JWT

在真实的企业级项目中，用户登录成功后，系统不能只返回一个简单的“登录成功”提示，更需要给前端一个可持续使用的身份凭证，使用户在后续请求中能够被系统正确识别。与此同时，前后端分离项目通常采用无状态方式管理登录状态，这就要求后端能够签发 Token，前端能够保存 Token，请求时能够携带 Token，服务端还要能够解析并校验 Token。

本任务将实现 **JWT 登录鉴权与前后端联动**，下一次任务实现 **前端密码预处理提交**。

1. 添加 JWT 依赖

在 `pom.xml` 中加入：

```
<!-- JWT处理库依赖 -->
    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-api</artifactId>
        <version>0.12.6</version>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-impl</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>

    <dependency>
        <groupId>io.jsonwebtoken</groupId>
        <artifactId>jjwt-jackson</artifactId>
        <version>0.12.6</version>
        <scope>runtime</scope>
    </dependency>
```

2. 添加JWT配置

```
security:
  jwt:
    secret: demo-jwt-secret-key-for-spring-security-2026
    expiration-millis: 7200000
```

这是在 `application.yaml` 里写的自定义配置项，专门给后续 `JwtUtil` 读取使用

- `security.jwt.secret`表示JWT 的签名密钥：

JWT 生成时，后端会用这个密钥去“签名”token；

JWT 解析时，也会用同一个密钥去校验 token 是不是合法的、有没有被篡改。

- `security.jwt.expiration-millis`表示 JWT 的有效时间，单位是毫秒，7200000=2小时。

3.新建security文件夹，编写相关安全组件

新建文件夹：`src/main/java/com/example/demo/security`，专门用于存放Spring Security鉴权相关。

3.1 在security目录下新建JwtUtil.java工具类

`JwtUtil` 编写用于JWT的工具类，函数用于处理：

- 生成 JWT
- 解析 JWT
- 校验 JWT 是否过期、是否属于当前用户

```
@Component
public class JwtUtil {

    @Value("${security.jwt.secret}")
    private String secret;

    @Value("${security.jwt.expiration-millis}")
    private long expirationMillis;
```

```

/**
 * 根据配置文件中的 secret 生成签名密钥
 */
private SecretKey getSignKey() {
    return Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
}

/**
 * 生成 JWT
 */
public String generateToken(String username) {
    Date now = new Date();
    Date expireDate = new Date(now.getTime() + expirationMillis);

    return Jwts.builder()
        .subject(username) // 主题：这里存用户名
        .issuedAt(now) // 签发时间
        .expiration(expireDate) // 过期时间
        .signWith(getSignKey()) // 签名
        .compact();
}

/**
 * 解析全部 Claims
 */
public Claims extractClaims(String token) {
    return Jwts.parser()
        .verifyWith(getSignKey())
        .build()

```

3.2 在security文件夹下新建JwtAuthenticationFilter.java

`JwtAuthenticationFilter` 是一个JWT认证过滤器，处理“登录成功后，后续请求如何识别用户身份”，用于：

- 从请求中取出 token
- 调用 `JwtUtil`
- 把认证信息放入 Security 上下文

JWT认证过滤器流程如下：

1. 每次请求到来时，先检查请求头中是否携带 JWT
2. 如果携带了 Bearer Token，就解析并校验 JWT
3. 如果 JWT 合法，则将当前用户的认证信息放入 SecurityContextHolder
4. 这样 Spring Security 就能识别当前请求已登录

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Autowired
    private JwtUtil jwtUtil;

    @Autowired
    private UserMapper userMapper;

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        // 1. 读取请求头中的 Authorization
        String authHeader = request.getHeader("Authorization");

        // 2. 如果没有 Authorization，或者不是 Bearer 开头，直接放行给后续过滤器
        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
            filterChain.doFilter(request, response);
            return;
        }

        // 3. 截取真正的 JWT 字符串
        String jwt = authHeader.substring(7);

        String username;

        try {
            // 4. 从 JWT 中解析用户名
            username = jwtUtil.extractUsername(jwt);
        } catch (Exception e) {
            // token 解析失败，直接继续后续过滤器
            filterChain.doFilter(request, response);
            return;
        }
    }
}
```

```
}
```

```
// 5. 如果解析到了用户名，并且当前还没有认证信息
```

4.修改业务逻辑

4.1 修改 UserServiceImpl 的 login() 方法

注入jwtUtil

```
@Autowired  
private JwtUtil jwtUtil;
```

将原先的代码

```
String token = "Bearer " + UUID.randomUUID().toString().replace("-", "");
```

修改为如下，登录成功后，调用JwtUtil工具类生成jwt令牌返回：

```
String jwt = jwtUtil.generateToken(userDTO.getUsername());  
return Result.success(jwt);
```

4.2 修改 SecurityConfig

SecurityConfig 中添加注入 JwtAuthenticationFilter，使得其加到用户名密码过滤器前。

增加注入：

```
@Autowired  
private JwtAuthenticationFilter jwtAuthenticationFilter;
```

并在 securityFilterChain 中添加：

```
.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.
```

```
http
    .cors(Customizer.withDefaults())
    .csrf(AbstractHttpConfigurer::disable)
    .sessionManagement(session ->
        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
    )
    .authorizeHttpRequests(auth -> auth
        .requestMatchers(HttpMethod.POST, "/api/users").permitAll()
        .requestMatchers(HttpMethod.POST, "/api/users/login").permitAll()
        .anyRequest().authenticated()
    )
    .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
    .formLogin(AbstractHttpConfigurer::disable)
    .httpBasic(AbstractHttpConfigurer::disable);
```

1.1 验证登录接口获取 JWT

```
{  
  "code": 200,  
  "data": "eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJhZG1pbGlzImVhdCI6MTc0NTIwNzkzMWwiZXh0LCJmYXRlICJoZWkiOjEwfQ.eyJzdWIiOiJhZG1pbGlzImVhdCI6MTc0NTIwNzkzMWwiZXh0LCJmYXRlICJoZWkiOjEwfQ.  
  "msg": "操作成功"  
}
```

1.2 携带 JWT 访问受保护接口

验证 JWT 是否能作为登录凭证，访问需要认证的接口。

方式一：标准做法（推荐使用 Postman 自带的 Auth 面板）

1. 在请求 URL 下方的选项卡中，点击 **Authorization**。
2. 在 **Type** 下拉菜单中，选择 **Bearer Token**。
3. 在右侧出现的 **Token** 输入框中，直接粘贴你刚才登录获取的 JWT 字符串。注意：这里只需要粘贴 `eyJ...` 开头的密文，不要自己手敲 `Bearer`，Postman 会自动帮你拼接好。

方式二：硬核做法（手动设置 Header，更贴近底层原理）

1. 切换到 **Headers** 选项卡。
2. 在 **Key** 这一列输入：`Authorization`
3. 在 **Value** 这一列输入：`Bearer 你的JWT字符串` *(注意：`Bearer` 和后面的密文之间必须有一个空格！例如 `Bearer eyJhbG...`)*。

观察是否能够访问之前禁止的接口。

提交要求

1. **测试与任务成功示例：**
截取SpringSecurity 中携带JWT访问成功接口的对比测试结果图。
 2. **文档提交：**将更新后的 `SecurityConfig` `JwtAuthenticationFilter` `JwtUtil` 核心代码推送至个人仓库，并将截图粘贴至 Word 文档按时提交。
-